

PostgreSQL：AI 时代最适合的数据库 已付费

原创 神说要有光 神光的幸福生活 2026年5月30日 23:37 山东

关系型数据库是互联网应用的基石。

账号信息、订单数据、聊天记录，或者企业的业务数据，几乎全部都依赖关系型数据库存储。

你用豆包、gemini 之类的 agent 的时候，不管多久的会话、聊天，都能翻到记录：

这也是存在关系型数据库里的。

一般是这样的表结构：

三个表：用户表存用户信息，会话表存左侧会话列表的消息，消息表存具体的消息

当用户登录的时候，会查出所有的会话列表显示在左边，这用到表和表的关联查询，用户和会话是一对多关系

当点击某个会话的时候，会查询所有的历史消息，会话和消息也是一对多关系

id 是主键 (primary key)，用于表示表的一条记录 (record)

user_id、conversation_id 是外键 (foreign key)，用于关联其他表的主键

通过这种主外键就可以实现表和表的关联查询

比如 sql 语句如下：

```
// 1. 根据用户 ID 查询他的所有会话
SELECT *
FROM conversations
WHERE user_id = '你的用户ID';

// 2. 根据会话 ID 查询这个会话里的所有消息
SELECT *
FROM messages
WHERE conversation_id = '你的会话ID'
ORDER BY created_at ASC;
```

MySQL、PostgreSQL（简称 PG）都是很流行的关系型数据库。

但在 AI 时代，PostgreSQL 优势更大。

因为消息内容需要加一个对应的向量字段用于语义检索：

mysql 是不支持的，你需要在 milvus 里建一个对应的集合：

这里用同样的结构来创建 milvus 集合就行，id 和 message.id 一致。

这样语义检索出数据后，可以关联到 MySQL 那边。

查询的时候是这样，写入的时候也要写两份，同样的数据要双写到 MySQL + Milvus ，比较麻烦。

那如果关系型数据库也支持向量检索就好了。

没错，这就是 PostgreSQL 的最大优势。

PostgreSQL 只需要在原本的消息表上，多加一个向量字段，不需要额外的数据库，不需要双写，不需要维护两套系统。

所有消息、会话、用户数据、向量语义特征，全部存在同一张表里。

查询的时候更简单。不用先查 Milvus、再查 MySQL，再手动拼接结果。

一条 SQL 就能同时做到：

```
-- AI 长期记忆：根据用户ID + 语义检索历史消息
SELECT m.*
FROM messages m
JOIN conversations c
ON m.conversation_id = c.id
WHERE
    c.user_id = '你的用户ID' -- 只查这个用户
AND c.id = '你的会话ID' -- 只查这个会话
ORDERBY
    m.embedding <=> '[1.2, 0.5, 0.8, ...]' -- 向量相似度检索
LIMIT5;
```

按用户过滤、按会话筛选、按时间排序、按语义检索。

这就是 AI 时代最需要的能力。

业务关系 + 向量检索，完美融合。

不用拆分架构，不用同步数据，不用写复杂的关联逻辑。

一张表，搞定传统关系查询 + AI 长期记忆。

所以你会发现 OpenAI、豆包、Kimi、通义千问、Dify 这些头部 AI 产品，几乎都把 PostgreSQL 当成核心数据库。

不是 MySQL 不好，而是在 AI 时代 PostgreSQL 真的太合适了。

这节课我们就来学一下 PostgreSQL

```
mkdir pgsql-test
cd pgsql-test
npm init -y
```

创建 docker-compose.yml

```
services:
  # PostgreSQL with pgvector (AI 向量数据库)
  postgres:
    image:pgvector/pgvector:pg16
    container_name:pg_vector_db
    restart:always
    environment:
      POSTGRES_USER:user
      POSTGRES_PASSWORD:123456
      POSTGRES_DB:hello_pg
    ports:
      -"5432:5432"
    volumes:
      -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/postgres:/var/lib/postgresql/data
      -./init-scripts:/docker-entrypoint-initdb.d
    healthcheck:
```

```
    test:["CMD-SHELL","pg_isready -U user -d hello_pg"]
    interval:5s
    timeout:5s
    retries:5

# PostgreSQL GUI (pgAdmin)
pgadmin:
  container_name:pgadmin
  image:dpage/pgadmin4:latest
  environment:
    PGADMIN_DEFAULT_EMAIL:admin@admin.com
    PGADMIN_DEFAULT_PASSWORD:admin
  volumes:
    -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/pgadmin:/var/lib/pgadmin
  healthcheck:
    test:["CMD","curl","-f","http://localhost:80/login"]
    interval:30s
    timeout:20s
    retries:3
  ports:
    -"8088:80"
  depends_on:
    -postgres

networks:
  default:
    name:common-network
```

跑一下:

```
docker compose up -d
```

神光的编程秘籍

接下来创建下表：

create_tables.sql

```

-- 启用 pgvector 向量扩展
CREATE EXTENSION IFNOTEXISTS vector;

-- 用户表
CREATETABLEIFNOTEXISTSusers (
    idSERIAL PRIMARY KEY,
    nameTEXTNOTNULL,
    created_at TIMESTAMPTIME WITH TIME ZONE DEFAULTCURRENT_TIMESTAMP
);

-- 会话表
CREATETABLEIFNOTEXISTS conversations (
    idSERIAL PRIMARY KEY,
    user_id INTEGERNOTNULL,
    title TEXT,
    created_at TIMESTAMPTIME WITH TIME ZONE DEFAULTCURRENT_TIMESTAMP,
    CONSTRAINT fk_conversations_user
        FOREIGNKEY (user_id) REFERENCESusers(id)
        ONDELETECASCADE
);

-- 消息表 (带向量)
CREATETABLEIFNOTEXISTS messages (
    idSERIAL PRIMARY KEY,
    conversation_id INTEGERNOTNULL,
    roleTEXTNOTNULLCHECK (roleIN ('user', 'assistant', 'system')),
    contentTEXTNOTNULL,
    embedding vector(1024), -- 与 EMBEDDING_MODEL 输出维度一致 (text-embedding-v3 为 1024)
    created_at TIMESTAMPTIME WITH TIME ZONE DEFAULTCURRENT_TIMESTAMP,
    CONSTRAINT fk_messages_conversation
        FOREIGNKEY (conversation_id) REFERENCESconversations(id)
        ONDELETECASCADE
);

-- 向量索引 (加速搜索)
CREATEINDEXIFNOTEXISTS idx_messages_embedding
    ON messages USING hnsw (embedding vector_cosine_ops);

```

跑一下:

我们把 sql 移到了 init-scripts 目录下，这样 pg 容器启动就会自动执行建表语句

接下来就可以增删改查了。

因为有个向量字段，需要用嵌入模型来生成向量，我们用代码来做 crud：

```
pnpm install pg @langchain/openai dotenv
```

先创建 .env

```
DATABASE_URL=postgresql://user:123456@localhost:5432/hello_pg
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
OPENAI_API_KEY=sk-xx
```

然后分别写下三个表的 CRUD 代码：

src/db.mjs

```
import "dotenv/config";
import pg from "pg";

const { Pool } = pg;

const pool = new Pool({
  connectionString: process.env.DATABASE_URL
});

async function query(text, params) {
  return pool.query(text, params);
}
```

```
export { pool, query };
```

用 pg 这个包连接数据库，创建 Pool，用 pool.query 执行 sql

src/users.mjs

```

import { query } from "../db.mjs";

asyncfunction createUser(name) {
  const { rows } = await query(
    "INSERT INTO users (name) VALUES ($1) RETURNING *",
    [name]
  );
  return rows[0];
}

asyncfunction getUserById(id) {
  const { rows } = await query("SELECT * FROM users WHERE id = $1", [id]);
  return rows[0] ?? null;
}

asyncfunction getAllUsers() {
  const { rows } = await query("SELECT * FROM users ORDER BY id");
  return rows;
}

asyncfunction updateUser(id, name) {
  const { rows } = await query(
    "UPDATE users SET name = $1 WHERE id = $2 RETURNING *",
    [name, id]
  );
  return rows[0] ?? null;
}

asyncfunction deleteUser(id) {
  const { rowCount } = await query("DELETE FROM users WHERE id = $1", [id]);
  return rowCount > 0;
}

export {
  createUser,
  getUserById,
  getAllUsers,
  updateUser,
  deleteUser,
};

```

用户表的 CRUD 代码

src/conversations.mjs

```

import { query } from "../db.mjs";

asyncfunction createConversation(userId, title = null) {
  const { rows } = await query(
    "INSERT INTO conversations (user_id, title) VALUES ($1, $2) RETURNING *",
    [userId, title]
  );
  return rows[0];
}

asyncfunction getConversationById(id) {
  const { rows } = await query(
    "SELECT * FROM conversations WHERE id = $1",
    [id]
  );
  return rows[0] ?? null;
}

asyncfunction getConversationsByUserId(userId) {
  const { rows } = await query(
    "SELECT * FROM conversations WHERE user_id = $1 ORDER BY created_at DESC",
    [userId]
  );
  return rows;
}

asyncfunction getAllConversations() {
  const { rows } = await query(
    "SELECT * FROM conversations ORDER BY created_at DESC"
  );
  return rows;
}

asyncfunction updateConversation(id, { title }) {
  const { rows } = await query(
    "UPDATE conversations SET title = $1 WHERE id = $2 RETURNING *",
    [title, id]
  );
  return rows[0] ?? null;
}

asyncfunction deleteConversation(id) {
  const { rowCount } = await query(
    "DELETE FROM conversations WHERE id = $1",
    [id]
  );
  return rowCount > 0;
}

```

```

}

export {
  createConversation,
  getConversationById,
  getConversationsByUserId,
  getAllConversations,
  updateConversation,
  deleteConversation,
};

```

对话表的 CRUD 代码

还有消息表的 CRUD 代码

src/messages.mjs

```

import "dotenv/config";
import { OpenAIEmbeddings } from "@langchain/openai";
import { query } from "../db.mjs";

const VALID_ROLES = ["user", "assistant", "system"];

let embeddings;

function getEmbeddings() {
  if (!embeddings) {
    embeddings = new OpenAIEmbeddings({
      model: process.env.EMBEDDING_MODEL || "text-embedding-v3",
      apiKey: process.env.OPENAI_API_KEY,
      configuration: {
        baseUrl: process.env.OPENAI_BASE_URL,
      },
    });
  }
  return embeddings;
}

async function createMessage(conversationId, role, content, withEmbedding = false) {
  if (!VALID_ROLES.includes(role)) {
    throw new Error(`role 必须是 ${VALID_ROLES.join(", ")} 之一`);
  }

  if (withEmbedding) {
    const vector = await getEmbeddings().embedQuery(content);
    const { rows } = await query(
      `INSERT INTO messages (conversation_id, role, content, embedding)`
    );
  }
}

```

```

        VALUES ($1, $2, $3, $4::vector)

        RETURNING id, conversation_id, role, content, created_at`,
    [conversationId, role, content, JSON.stringify(vector)]
  );
  return rows[0];
}

const { rows } = await query(
  `INSERT INTO messages (conversation_id, role, content)
  VALUES ($1, $2, $3)
  RETURNING *`,
  [conversationId, role, content]
);
return rows[0];
}

asyncfunction getMessageById(id) {
  const { rows } = await query(
    `SELECT id, conversation_id, role, content, created_at
    FROM messages WHERE id = $1`,
    [id]
  );
  return rows[0] ?? null;
}

asyncfunction getMessagesByConversationId(conversationId) {
  const { rows } = await query(
    `SELECT id, conversation_id, role, content, created_at
    FROM messages
    WHERE conversation_id = $1
    ORDER BY created_at ASC`,
    [conversationId]
  );
  return rows;
}

asyncfunction updateMessage(id, content, withEmbedding = false) {
  if (withEmbedding) {
    const vector = await getEmbeddings().embedQuery(content);
    const { rows } = await query(
      `UPDATE messages
      SET content = $1, embedding = $2::vector
      WHERE id = $3
      RETURNING id, conversation_id, role, content, created_at`,
      [content, JSON.stringify(vector), id]
    );
    return rows[0] ?? null;
  }

```

```

    }

    const { rows } = await query(
      `UPDATE messages SET content = $1 WHERE id = $2 RETURNING *`,
      [content, id]
    );
    return rows[0] ?? null;
  }

  asyncfunction deleteMessage(id) {
    const { rowCount } = await query("DELETE FROM messages WHERE id = $1", [id]);
    return rowCount > 0;
  }

  asyncfunction searchSimilarMessages(conversationId, searchText, limit = 5) {
    const vector = await getEmbeddings().embedQuery(searchText);
    const { rows } = await query(
      `SELECT id, conversation_id, role, content, created_at,
        1 - (embedding <=> $1::vector) AS similarity
      FROM messages
      WHERE conversation_id = $2 AND embedding IS NOT NULL
      ORDER BY embedding <=> $1::vector
      LIMIT $3`,
      [JSON.stringify(vector), conversationId, limit]
    );
    return rows;
  }

  export {
    createMessage,
    getMessageById,
    getMessagesByConversationId,
    updateMessage,
    deleteMessage,
    searchSimilarMessages,
  };

```

这个要用到嵌入模型来做向量化

然后在 src/index.mjs 里用一下：

```

import { pool } from "../db.mjs";
import * as users from "../users.mjs";
import * as conversations from "../conversations.mjs";
import * as messages from "../messages.mjs";

asyncfunction run() {

```

```
console.log("=== 用户 CRUD ===");

const user = await users.createUser("张三");
console.log("创建用户:", user);

const fetchedUser = await users.getUserById(user.id);
console.log("查询用户:", fetchedUser);

const updatedUser = await users.updateUser(user.id, "李四");
console.log("更新用户:", updatedUser);

console.log("\n=== 会话 CRUD ===");

const conversation = await conversations.createConversation(
  user.id,
  "第一次对话"
);
console.log("创建会话:", conversation);

const userConversations = await conversations.getConversationsByUserId(
  user.id
);
console.log("用户的会话列表:", userConversations);

const updatedConversation = await conversations.updateConversation(
  conversation.id,
  { title: "更新后的标题" }
);
console.log("更新会话:", updatedConversation);

console.log("\n=== 消息 CRUD ===");

const userMessage = await messages.createMessage(
  conversation.id,
  "user",
  "你好, 请介绍一下 PostgreSQL"
);
console.log("创建用户消息:", userMessage);

const assistantMessage = await messages.createMessage(
  conversation.id,
  "assistant",
  "PostgreSQL 是一个功能强大的开源关系型数据库。"
);
console.log("创建 AI 消息:", assistantMessage);

const conversationMessages = await messages.getMessagesByConversationId(
  conversation.id
```

```

    );
    console.log("会话消息列表:", conversationMessages);

    const updatedMessage = await messages.updateMessage(
        userMessage.id,
        "你好, 请介绍一下 pgvector"
    );
    console.log("更新消息:", updatedMessage);

    console.log("\n=== 语义检索 ===");

    const seedMessages = [
        { role: "user", content: "PostgreSQL 支持哪些数据类型? " },
        {
            role: "assistant",
            content:
                "PostgreSQL 支持整数、文本、JSON、数组, 以及 pgvector 扩展提供的向量类型。",
        },
        { role: "user", content: "怎么做相似度搜索? " },
        {
            role: "assistant",
            content:
                "可以使用 pgvector 的 cosine 距离运算符 <=>, 配合 hnsw 索引加速向量检索。",
        },
    ];

    for (const msg of seedMessages) {
        await messages.createMessage(
            conversation.id,
            msg.role,
            msg.content,
            true
        );
    }

    console.log(`已写入 ${seedMessages.length} 条带 embedding 的消息`);

    const searchQueries = ["向量相似度怎么查", "关系型数据库有哪些类型"];

    for (const searchText of searchQueries) {
        console.log(`\n搜索: "${searchText}"`);
        const results = await messages.searchSimilarMessages(
            conversation.id,
            searchText,
            3
        );
        if (results.length === 0) {
            console.log(" 无匹配结果");
            continue;
        }
    }

```

```

    }
    for (const [i, row] of results.entries()) {
      console.log(
        ` ${i + 1}. [${row.role}] ${row.content} (similarity: ${Number(row.similarity).toFixed(2)})`
      );
    }
  }
}

// console.log("\n=== 清理 ===");

// await messages.deleteMessage(assistantMessage.id);
// await messages.deleteMessage(updatedMessage.id);
// await conversations.deleteConversation(conversation.id);
// await users.deleteUser(user.id);

// console.log("演示数据已清理");
}

run()
  .catch((err) => {
    console.error("运行失败:", err.message);
    process.exit(1);
  })
  .finally(() => pool.end());

export { users, conversations, messages };

```

跑下试试：

```
node src/index.mjs
```

重点是语义检索这部分：

核心就是根据传入的向量和这个向量字段做相似度判断，排序后取前几条就可以了。

其次是多表的关联查询：

查询某个用户的所有会话：

某个会话的所有消息：

当然，这些 sql 不需要记住，大概理解就行，我们一般都是通过 ORM 框架才操作数据库。

比如前面讲过的 TypeORM。

创建 nest 项目：

```
nest new typeorm-pg-crud
```

安装依赖：

```
pnpm install --save @nestjs/typeorm typeorm pg
```

在 AppModule 引入 typeorm：

```
TypeOrmModule.forRoot({  
  type: 'postgres',  
  host: 'localhost',  
  port: 5432,  
  username: 'user',  
  password: '123456',  
  database: 'hello_pg',  
  synchronize: true,  
  logging: true,  
  entities: []  
})
```

指定数据库连接信息、database

然后分别创建 conversations 模块

```
nest g res conversations --no-spec
```

生成 CRUD 代码

现在只有 conversation 的 Entity:

我们补全 entity:

entities/user.entity.ts

```
import {
  Column,
  CreateDateColumn,
  Entity,
  OneToMany,
  PrimaryGeneratedColumn,
} from 'typeorm';
import { Conversation } from '../conversation.entity';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ type: 'text' })
```

```

name: string;

@CreateDateColumn({ type: 'timestampz', name: 'created_at' })
createdAt: Date;

@OneToMany(() => Conversation, (conversation) => conversation.user)
conversations: Conversation[];
}

```

entities/conversation.entity.ts

```

import {
  Column,
  CreateDateColumn,
  Entity,
  JoinColumn,
  ManyToOne,
  OneToMany,
  PrimaryGeneratedColumn,
} from 'typeorm';
import { User } from '../user.entity';
import { Message } from '../message.entity';

@Entity('conversations')
export class Conversation {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ name: 'user_id' })
  userId: number;

  @Column({ type: 'text', nullable: true })
  title: string | null;

  @CreateDateColumn({ type: 'timestampz', name: 'created_at' })
  createdAt: Date;

  @ManyToOne(() => User, (user) => user.conversations, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'user_id' })
  user: User;

  @OneToMany(() => Message, (message) => message.conversation)
  messages: Message[];
}

```

entities/message.entity.ts

```

import {
  Column,
  CreateDateColumn,
  Entity,
  JoinColumn,
  ManyToOne,
  PrimaryGeneratedColumn,
} from 'typeorm';
import { Conversation } from './conversation.entity';

export enum MessageRole {
  USER = 'user',
  ASSISTANT = 'assistant',
  SYSTEM = 'system',
}

@Entity('messages')
export class Message {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ name: 'conversation_id' })
  conversationId: number;

  @Column({
    type: 'text',
    enum: MessageRole,
  })
  role: MessageRole;

  @Column({ type: 'text' })
  content: string;

  @Column('vector', { length: 1024, nullable: true })
  embedding: number[] | null;

  @CreateDateColumn({ type: 'timestampz', name: 'created_at' })
  createdAt: Date;

  @ManyToOne(() => Conversation, (conversation) => conversation.messages, {
    onDelete: 'CASCADE',
  })
  @JoinColumn({ name: 'conversation_id' })
  conversation: Conversation;
}

```

这里主要是一对多关系的映射，需要用到 `@OneToMany`、`@ManyToOne` 的装饰器

神光的编程秘籍

做好了表、列、一对多关系的映射

在 `entities` 数组里引入下：

这样我们就可以用 `typeorm` 做三个实体的 `crud` 了

语义检索要用到嵌入模型，安装下依赖：

```
pnpm install @langchain/openai dotenv
```

创建 .env

```
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
OPENAI_API_KEY=sk-xx
```

改一下 conversations.service.ts

```
import 'dotenv/config';
import {
  BadRequestException,
  Injectable,
  NotFoundException,
} from '@nestjs/common';
import { InjectEntityManager } from '@nestjs/typeorm';
import { OpenAIEmbeddings } from '@langchain/openai';
import { EntityManager } from 'typeorm';
import { User } from '../entities/user.entity';
import { Conversation } from '../entities/conversation.entity';

export interface SemanticSearchResult {
  id: number;
  conversation_id: number;
  role: string;
  content: string;
  created_at: Date;
  similarity: number;
}

@Injectable()
export class ConversationsService {
  private embeddings: OpenAIEmbeddings | null = null;

  constructor(
    @InjectEntityManager()
    private readonly em: EntityManager,
  ) {}

  /** 用户 → 会话 (一对多) */
  async findConversationsByUserId(userId: number) {
    const user = await this.em.findOne(User, {
      where: { id: userId },
      relations: { conversations: true },
      order: { conversations: { createdAt: 'DESC' } },
    });
```

```

});

if (!user) {
    thrownew NotFoundException(`User #${userId} not found`);
}

return user;
}

/** 会话 → 消息 (一对多) */
async findMessagesByConversationId(conversationId: number) {
    const conversation = awaitthis.em.findOne(Conversation, {
        where: { id: conversationId },
        relations: { messages: true },
        order: { messages: { createdAt: 'ASC' } } },
    });

    if (!conversation) {
        thrownew NotFoundException(`Conversation #${conversationId} not found`);
    }

    return {
        id: conversation.id,
        userId: conversation.userId,
        title: conversation.title,
        createdAt: conversation.createdAt,
        messages: conversation.messages.map(
            ({ id, conversationId, role, content, createdAt }) => ({
                id,
                conversationId,
                role,
                content,
                createdAt,
            }),
        ),
    };
}

/** 会话内语义检索 (pgvector 余弦距离) */
async searchSimilarMessages(
    conversationId: number,
    searchText: string,
    limit = 5,
): Promise<SemanticSearchResult[]> {
    const conversation = awaitthis.em.findOne(Conversation, {
        where: { id: conversationId },
    });

```

```

    if (!conversation) {
        thrownew NotFoundException(`Conversation #${conversationId} not found`);
    }

    const vector = awaitthis.embedQuery(searchText);

    const rows: SemanticSearchResult[] = awaitthis.em.query(
        `SELECT id, conversation_id, role, content, created_at,
           1 - (embedding <=> $1::vector) AS similarity
        FROM messages
        WHERE conversation_id = $2 AND embedding IS NOT NULL
        ORDER BY embedding <=> $1::vector
        LIMIT $3`,
        [JSON.stringify(vector), conversationId, limit],
    );

    return rows.map((row) => ({
        ...row,
        similarity: Number(row.similarity),
    }));
}

private getEmbeddings(): OpenAIEmbeddings {
    if (!this.embeddings) {
        if (!process.env.OPENAI_API_KEY) {
            thrownew BadRequestException(
                '语义检索需要配置 OPENAI_API_KEY (与 pgsql-test 相同)',
            );
        }
        this.embeddings = new OpenAIEmbeddings({
            model: process.env.EMBEDDING_MODEL || 'text-embedding-v3',
            apiKey: process.env.OPENAI_API_KEY,
            configuration: {
                baseUrl: process.env.OPENAI_BASE_URL,
            },
        });
    }
    returnthis.embeddings;
}

private async embedQuery(text: string): Promise<number[]> {
    returnthis.getEmbeddings().embedQuery(text);
}
}

```

这里实现了查询用户的所有会话、查询某个会话的所有消息的关联查询。

只要加上 relations 就可以关联查询了：

要注意的是向量检索是扩展的 sql 语法，所以得用 sql 写查询：

流程和之前一样。

然后改下 controller 加一下三个接口：

```
import {
  Body,
  Controller,
  DefaultValuePipe,
  Get,
  Param,
  ParseIntPipe,
  Post,
  Query,
} from '@nestjsjs/common';
import { ConversationsService } from '../conversations.service';
import { SemanticSearchDto } from '../dto/semantic-search.dto';

@Controller('conversations')
export class ConversationsController {
  constructor(private readonly conversationsService: ConversationsService) {}

  /** GET /conversations/users/:userId - 用户的会话列表 */
  @Get('users/:userId')
  findByUser(@Param('userId', ParseIntPipe) userId: number) {
    return this.conversationsService.findConversationsByUserId(userId);
  }

  /** GET /conversations/:id/messages - 会话的消息列表 */
```

```

@Get('/:id/messages')
findMessages(@Param('id', ParseIntPipe) id: number) {
  return this.conversationsService.findMessagesByConversationId(id);
}

/** POST /conversations/:id/search - 会话内语义检索 */
@Post('/:id/search')
search(
  @Param('id', ParseIntPipe) id: number,
  @Body() dto: SemanticSearchDto,
  @Query('limit', new DefaultValuePipe(5), ParseIntPipe) queryLimit?: number,
) {
  const limit = dto.limit ?? queryLimit ?? 5;
  return this.conversationsService.searchSimilarMessages(
    id,
    dto.query,
    limit,
  );
}
}

```

还要创建用到的接受参数的 dto

dto/semantic-search.dto.ts

```

export class SemanticSearchDto {
  query: string;
  limit?: number;
}

```

改下服务启动的端口：

跑一下：

```
pnpm run start:dev
```

我们准备一些 curl:

```
// 用户 → 会话 (一对多)
curl -s http://localhost:3005/conversations/users/2 | jq
// 会话 → 消息 (一对多)
curl -s http://localhost:3005/conversations/2/messages | jq
// 语义检索
curl -s -X POST http://localhost:3005/conversations/2/search \
  -H 'Content-Type: application/json' \
  -d '{"query": "向量相似度怎么查", "limit": 3}' | jq

curl -s -X POST 'http://localhost:3005/conversations/2/search?limit=5' \
  -H 'Content-Type: application/json' \
  -d '{"query": "PostgreSQL 支持哪些数据类型"}' | jq
```

试一下:

这样我们就实现了基于 ORM 实现 PostgreSQL 的一对多关联查询，以及向量语义检索。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

PostgreSQL 在 AI 时代比 MySQL 更有优势，它可以通过 pgvector 插件实现向量字段，以及语义检索。

我们可以在 sql 里关联多个表查询，并且做语义检索。

相比 MySQL + Milvus 结合的方式，简化了不少。

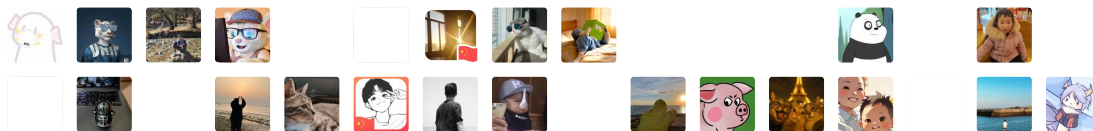
我们用 docker compose 跑了 PostgreSQL 和它的 UI 界面。

之后在 node 代码里通过 sql 做了 CRUD、语义检索。

并且又用 TypeORM + Nest 用 ORM 的方式实现了一对多关联查询，但语义检索还是得用 sql，因为是扩展语法。

至此，我们就用 PostgreSQL 可以在业务里面直接实现关联查询 + 语义检索了，不再需要 MySQL + Milvus。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

DeepAgents 实战：多 Agent 架构的深度调研助手

下一篇

Redis：实现 Agent 短期记忆存储的最佳方案

留言 8

写留言



\$pades K 重庆 6月1日

光哥，这个依赖好像用不了而且npm里面也装不了吧@langchain/postgres



神光的幸福生活 作者 6月1日

代码里没用这个。我看看 install 的依赖是不是写多了，去掉一下



神光的幸福生活 作者 6月1日

去掉了。其实你直接跑仓库代码就行



What are yo... 内蒙古 6月1日

课程内容主要就是通过微信公众号看是吧？



神光的幸福生活 作者 6月1日

对的，pc端用微信内置浏览器打开



我睡觉时不困 上海 6月1日

回复 神光的幸福生活：现在好像不能复制内容（复制代码），上周还可以复制的



神光的幸福生活 作者 6月1日

回复 我睡觉时不困：直接从仓库复制吧



Hw 云南 5月31日

直接用Supabase也可以

首评

